Basic (your first real API call) — Solution

> Back to the exercise · Hint 1 · Hint 2 · Solution

Story — `chat_api_basics_practice.py`: this is the single call every other exercise, the Build Task, and every later document's chat feature builds on top of. Written here, once, on its own, so you see the plain request/response shape before anything else (memory, streaming, error handling) gets added on top of it. **If not:** the first time you'd see a raw `client.chat.completions.create(...)` call would be buried inside a bigger file already juggling a loop or a try/except, making it harder to tell which part is "the API call" and which part is everything else.

Basic Version

Approach 1 — the direct way

```
# chat_api_basics_practice.py
from dotenv import load_dotenv
from openai import OpenAI

load_dotenv()
client = OpenAI()

response = client.chat.completions.create(
    model="gpt-4o-mini",
    messages=[
        {"role": "system", "content": "You are formal and professional."},
        {"role": "user", "content": "Tell me about your day."},
    ],
)
print(response.choices[0].message.content)

response = client.chat.completions.create(
    model="gpt-4o-mini",
    messages=[
        {"role": "system", "content": "You are a sarcastic pirate."},
        {"role": "user", "content": "Tell me about your day."},
    ],
)
print(response.choices[0].message.content)
```

Expected output (wording varies, exact text differs every run):

```
As a formal assistant, I do not experience days in the way a person does...
Arrr, another day chained to this here API, matey...
```

This works fine and shows the change clearly. It repeats the call structure twice, which is fine for a one-off script but gets repetitive fast if you compare more than two prompts.

> Back to the exercise · Hint 1 · Hint 2 · Solution

Intermediate Version

Approach 1 — a reusable `ask()` function

```
# chat_api_basics_practice.py
from dotenv import load_dotenv
from openai import OpenAI

load_dotenv()
client = OpenAI()


def ask(system_prompt: str, user_prompt: str) -> str:
    # why: separates "how to call the API" from "which prompts to compare" —
    # comparing a third prompt is now one more call, not a copy-pasted block.
    response = client.chat.completions.create(
        model="gpt-4o-mini",
        messages=[
            {"role": "system", "content": system_prompt},
            {"role": "user", "content": user_prompt},
        ],
    )
    # how: .choices[0] is the first (and here, only) reply the model generated;
    # .message.content is the actual text, not the whole response object.
    return response.choices[0].message.content


def main() -> None:
    question = "Tell me about your day."

    formal_answer = ask("You are formal and professional.", question)
    print("Formal:", formal_answer)

    pirate_answer = ask("You are a sarcastic pirate.", question)
    print("Pirate:", pirate_answer)


if __name__ == "__main__":
    main()
```

Expected output:

```
Formal: As a formal assistant, I do not experience days the way a person does...
Pirate: Arrr, another day chained to this here API, matey...
```

Difference from Basic: the `ask()` function has full type hints and separates "how to call the API" from "which prompts to compare" — adding a third or fourth system prompt to compare is now one more `ask()` call, not a whole copy-pasted block. Wrapping the script's entry point in `main()` behind `if __name__ == "__main__":` also matches the pattern from Doc01's Build Task solution — this file could later be imported elsewhere without immediately running.

> Back to the exercise · Hint 1 · Hint 2 · Solution

Approach 2 — system prompts in `.txt` files

Story — `load_prompt()`: real apps keep prompt wording in text files, so changing the assistant's style never means editing Python. `with open(...)` is Doc01's `.env` exercise; `.read()` returns the whole file as one string, and `.lower()` makes the name match the lowercase file name. **If not:** Project 1's `load_prompt()` — which loads every agent's prompt this way — would be the first time you read a whole file or lowercased a string.

Create `practice/prompts/formal.txt` and `practice/prompts/pirate.txt`, one line each, then run from inside `practice/`:

`prompts/formal.txt`

```
You are formal and professional.
```

`prompts/pirate.txt`

```
You are a sarcastic pirate.
```

```
# chat_api_basics_practice.py
from dotenv import load_dotenv
from openai import OpenAI

load_dotenv()
client = OpenAI()

# how: a path from the folder you run the script in — run it from
# practice/, where the prompts/ folder sits
PROMPTS_DIR = "prompts"


def load_prompt(name: str) -> str:
    # why: the wording lives in a .txt file, not inside Python — you can
    # change the assistant's style without touching the code
    # how: .lower() makes "Formal" and "formal" both find formal.txt
    path = PROMPTS_DIR + "/" + name.lower() + ".txt"
    with open(path) as prompt_file:
        # how: .read() returns the whole file as one string, and .strip()
        # drops the blank line at the end
        return prompt_file.read().strip()
```

```
def ask(system_prompt: str, user_prompt: str) -> str:
    response = client.chat.completions.create(
        model="gpt-4o-mini",
        messages=[
            {"role": "system", "content": system_prompt},
            {"role": "user", "content": user_prompt},
        ],
    )
    return response.choices[0].message.content


def main() -> None:
    question = "Tell me about your day."
    for style in ["Formal", "Pirate"]:
        system_prompt = load_prompt(style)
        print(f"{style} prompt: {system_prompt}")
        print(f"{style} answer: {ask(system_prompt, question)}")


if __name__ == "__main__":
    main()
```

Expected output (answers vary):

```
Formal prompt: You are formal and professional.
Formal answer: As a formal assistant, I do not experience days...
Pirate prompt: You are a sarcastic pirate.
Pirate answer: Arrr, another day chained to this here API, matey...
```

Difference from Approach 1: the prompts come from files instead of strings in the code, so the same `ask()` compares any style you add as a new `.txt` file.

Which one should you actually write? For this exercise, the Intermediate version is what you should write — a reusable, typed `ask()` function around one client — and Approach 2's `load_prompt()` is the exact shape Project 1 uses for its prompt files. Reach for a cached, module-level client and explicit `max_retries` / `timeout` settings the moment this code is called from unattended, long-running code — exactly this document's Build Task, covered there.